

平行資料庫中動態負載平衡於排序運算之研究

Dynamic Load Balancing for Sorting Operation in Parallel Databases

羅有隆

Yu-lung Lo

陳秀娟

Shiou-jiuan Chen

黃渝臻

Yu-chen Huang

朝陽科技大學 資訊管理系
台中縣霧峰鄉吉峰東路 168 號
yllo@cyut.edu.tw

摘要

排序是常被運用在資料庫處理中的運算，例如在結構化查詢語言中之「依關鍵字將資料由小(大)到大(小)排列」與「去除重覆性的資料」的查詢語法上，都是需要藉助排序的處理。完全不共享多處理器系統架構是被公認為最具擴充性，且符合超大型資料庫的高產能需求，而查詢之資料排序是影響多處理器系統效能的重要因素之一。目前所被提出的平行資料排序演算法，皆假設所要排序的資料在所有處理器中的分配是均勻的，亦即每個處理器的負載是差不多，如此排序所需的執行時間是趨於理想化的。然而在實際的運用上，資料歪斜是常可能發生的，也就是每個處理器所分配到的資料並不均勻，如此擁有最大資料負擔的處理器，將是排序處理速度的瓶頸，整個系統效能亦可能隨之受到影響。此研究報告即在提出於多處理器資料庫中，利用動態負載平衡的方法，來分配資料做排序的運算，以降低因資料歪斜對系統效能所產生之負面影響。且經由實驗模擬的結果，我們發現，所提出的排序演算法有著令人振奮的效能改進。

1 簡介

隨著資料庫系統大小不斷地擴大，使得資料查詢變得更為複雜。特別是在一些超大型資料庫(Very Large DataBases, VLDB)應用系統中(如全國戶政系統、地理資訊系統、決策資訊系統等等)的查詢，經常是相當耗時的。因此，如何加速資料庫查詢的處理速度，是一重要且急需解決的問題。平行處理是可用來加速查詢處理的重要方法之一，由於微處理機的技術成熟且價格越來越便宜，使用多個處理器於資料庫系統中變得更加經

濟可行[1]。許多利用平行處理原理的多處理器資料庫系統原型早已被發展出來，典型的例子包括了東京大學的 Grace Machine[13]、Teradata 的 DBC/1012 資料庫電腦[19]、MCC 的 Bubba 系統[4]、以及威斯康辛大學的 GAMMA 資料庫系統[5]等。而商業化系統亦已不再少見。

而多處理器系統中，完全不共享(Shared-nothing)多處理器系統架構[16][17]被公認為最具擴充性，且符合超大型資料庫的高產能需求[5][6][13][19]。在此架構中，每個處理節點(Processing Node, PN)包含有一或數個處理器，以及擁有專屬的記憶體與硬碟，而這些處理節點靠著系統內高頻寬的網路連結。資料的運算，將在其所屬之處理節點中完成，而系統效能在此多處理器架構中，將取決於有效資源的利用與每一處理節點之查詢資料負載的平衡。

所謂排序(Sort)，乃是指將一組資料依據其資料間的特性或是依據使用者的需求，將資料呈現規則性的由小到大或由大到小的順序排列，以方便資料的比對或呈獻。它常被運用在資料庫的查詢處理上[7]，例如在結構化查詢語言(Structure Query Language, SQL)中之 ORDER BY(依關鍵字將資料由小/大到大/小排列)與 DISTINCT(先將資料排序後，再去除重覆性的資料)的查詢語法上都是需要藉助排序的處理。基本上排序可以分為內部和外部排序[18]。

內部排序，乃是指針對記憶體中的資料按某種順序進行編排，把資料完全放在主記憶體內進行排序的方式。目前於平行資料庫中的資料排序，亦假設於此前提下，而此種排序方法對於少量資料也特別顯得有效，但是資料庫不斷擴大的現今，已漸不切實際需求。外部排序，則是指當資料量很多，無法一次全部載入每個處理節點的主記憶體內進行排序，而必須借用外部的儲存裝置，以分批次進行排序，而此種排序相當的耗時。到目前為止，有關外部排序的研究，在單一處理器的環境下，其技術也已相當的成熟[18]，然而在

多處理器環境下的執行，則過於理想化。雖然其運用平行處理的觀念及技術來執行，但卻也都假設了每個處理節點有著相同量的資料負擔[3][18][20]，而忽略了可能發生的資料歪斜(data skew)[21]情形。當每個處理節點所要執行排序的資料量不一致時，整個查詢所需的回應時間，將被擁有最大量資料要排序的處理節點所延遲。

對於平行資料庫的操作過程中可能產生的資料歪斜，連結(join)運算是最被廣泛討論的，也提出了很多的解決方案。當連結運算傾向於利用雜湊(hashing)與動態負載平衡(dynamic load balancing)來處理資料歪斜的問題時[8][9][10][11][12]，其解決辦法並無法完全套用於排序上，因連結的結果並不要求資料的順序性。至今解決平行排序運算時資料歪斜的情況發生，則尚未受到注目。因此在一個理想的外部排序的多處理器系統中，我們不僅希望能達到最大的平行度(maximum degree of parallelism)，以使用最多的處理節點來幫助加速。而且每個處理節點的工作負擔，在所要處理的資料分配上皆能達到所謂的負載平衡，以避免任何瓶頸的產生。

在本研究報告裡，我們將引用動態負載平衡的觀念，修改其演算法使其適用於平行排序操作，以促進多處理器系統之效能。本篇報告共有五節，除了第一節簡介外，其他章節安排如下：在第二節中將現有的平行排序技術做擇要的介紹；並將負載平衡的技術做簡單的說明；於第三節，我們提出了動態負載平衡排序技術，以解決資料歪斜的問題；而在第四節中我們設計實驗模擬，以檢視所提出技術的效率與實用可行；接著於最後一節，我們將做研究報告的結論。

2 現有之研究概況

以下我們就以全不共享平行資料庫系統為例，介紹幾個現有的平行排序演算法與其優缺點。而對於動態負載平衡的操作，也於本節末做說明。

2.1 現有的平行排序技術

(1) Parallel Merge-All Sort

是簡單且基本的傳統排序方法，被採用於 GAMMA 資料庫系統[6]與其他商業性的資料庫系統等。它利用到排序和合併(merge)兩個觀念，其方式為先在每個處理節點做局部排序(local sort)，然後將所有資料串列(data list)傳送給主處理節點做合併的動作，當然最後是由一個處理節點來處理。執行過程如圖 1 所示。

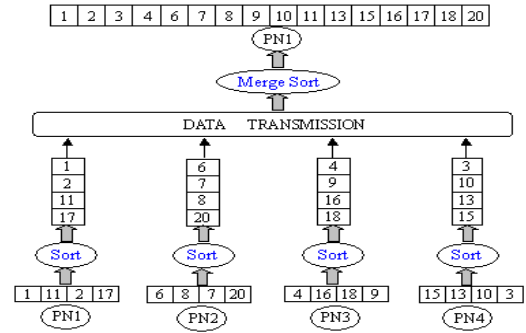


圖 1：Parallel Merge-All Sort

此演算法的優點是一個簡單的一階樹狀結構，容易設計。而缺點則是處理節點數很多時，主處理節點所需處理的資料量將相當的大，成為執行瓶頸，而其他的處理節點則閒置，又浪費了系統資源。

(2) Parallel Binary-Merge Sort

此排序技術是由 Bitton 等人提出的[2][3]，方法是每個處理節點先做局部排序，然後將每兩個處理節點中的資料做合併，結果留放於其中的一個處理節點，而每次合併僅需使用前一層一半的處理節點數，直到最後只剩下一個處理節點做最後的合併工作。執行過程如圖 2 所示。

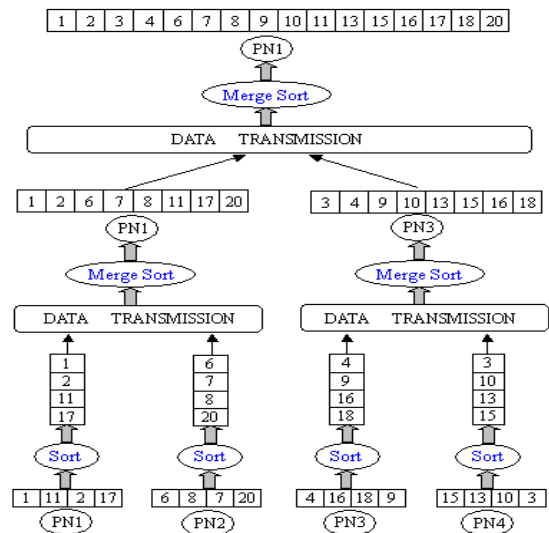


圖 2：Parallel Binary-Merge Sort

Parallel Binary-Merge Sort 的優點是合併過程可採 pipeline 方式來完成，而且是多數的處理節點都可參與合併的執行。缺點則是若處理節點數很多時，會產生較高的樹狀結構，而在 pipeline 越後面的處理節點所要處理的資料量則越多。如此最後的處理節點，仍然可能產生瓶頸，與 Parallel Merge-All Sort 有著類似的缺點。

(3) Parallel Partitioned Sort

Parallel Partitioned Sort 是由 Taniar 等人於 2000 年所提出的[18]。排序法分兩個階段，第一階段根據範圍分配(range distribution)的方式，來將資料重新分配到所有的處理節點，第二階段各個處理節點做局部的排序。如圖 3 所示，首先各處理節點讀出自己局部的資料(local data)，再依據資料所屬的範圍分別配送到各處理節點，此時資料是尚未排序的。資料配送完後，各處理節點為所分配到的資料做排序。

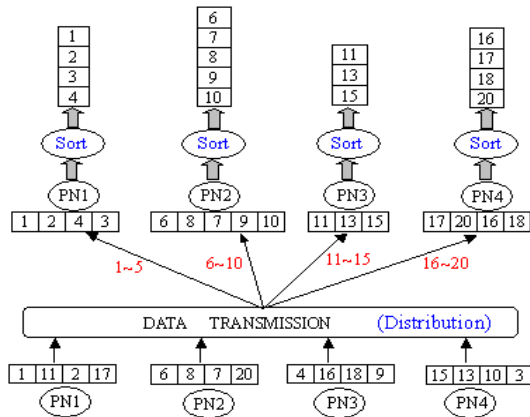


圖 3：Parallel Partition Sort

此排序法的優點是容易設計，在一般的情況下，資料重分配的結果，有可能使每個處理節點的負擔較為平均，且整個執行過程使用到所有的處理節點，達到最大的平行度，使排序執行較為有效率。然而缺點則是作者自己於報告中[18]所提的，資料以範圍等距分配如果

遇到資料歪斜，則並無法真正達到負載平衡的目的。

2.2 動態負載平衡(Dynamic Load Balancing)

綜觀以上的平行排序演算法，只有 Parallel Partition Sort 簡單的嘗試平衡分配處理節點的負擔，但是仍然沒有真正的解決資料歪斜的問題。在過去有關解決資料歪斜的問題，在處理兩資料表格的平行連結(join) 運算最佳化的研究上，有著相當不錯的成果，如[8][9][12]，以及本文作者所參與的報告[10][11]。而解決這些連結運算時所可能產生的資料歪斜，多採用所謂的動態負載平衡技術。其方法是每個處理節點以雜湊(hashing)將其局部資料分成許多的區塊(buckets)，並將區塊的大小(資料筆數)的訊息傳送給一協調處理器(coordinator)，此協調處理器統計所接收到每一區塊的大小資訊後，接著用 Best Fit Decreasing 策略，將這些區塊按大小做遞減的排序，再從資料最多的區塊開始分配給處理節點，而每次要分配

區塊給當時擁有被分配到最少資料的處理節點為原則，直到區塊分配完為止。如圖 4 將區塊分配給兩個處理節點為範例所示。此技術可以有效的改善資料分配不均的問題，進而提升執行的效能。

由於資料表格的連結運算若無特別要求，並不需要將資料排序輸出，因此動態負載平衡的技術，利用雜湊與區塊排序來分配資料給所有處理節點，對於平行連結運算解決資料歪斜的問題，有著相當不錯的成效。然而同樣技術並不完全適用於要求結果按一定順序排列的平行排序運算上，不過相同概念經適度的修改後，將是可行的，在下一節將介紹我們所提出的演算法。

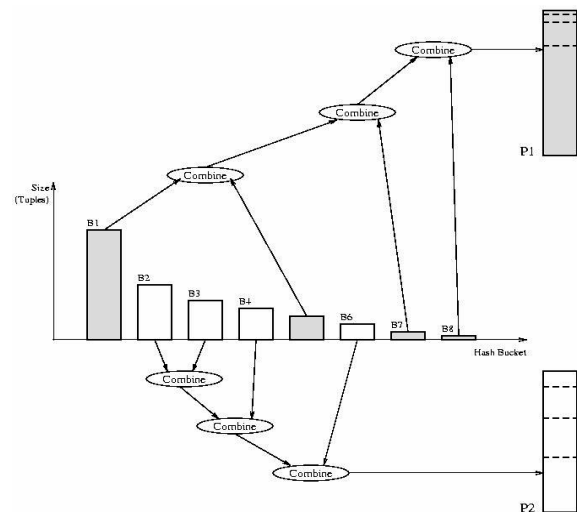


圖 4：Best Fit Decreasing Strategy

3 考量負載平衡的平行排序

在本節中，我們提出兩種考量負載平衡的平行排序演算法，分別為 Partitioned with Redistribution Sort 與 Load Balancing Optimization Sort，兩種演算法皆於執行的不同階段運用到重新修改後的動態負載平衡的技術，詳細分述如下：

3.1 Partitioned with Redistribution Sort (PRS)

PRS 是修改 Parallel Partitioned Sort 演算法而來，其再於資料以範圍分配後，若發現資料歪斜的發生，則在做排序之前，再做一次資料的重分配。然而為了有利於資料的重分配，PRS 也利用了動態負載平衡的概念，將資料以範圍分成很多的區塊，但並不作區塊的排序，整個演算法呈現於圖 5，並由以下四個階段(Phases)所完成：

Phase 1. Range Partitioned： 每個處理節點將局部所擁有的資料以範圍區分配送出去，同時每個處理節點也接收所被分配範圍內的

資料。而接收到的資料以更小範圍的區塊來存放。例如 PN1 如果被分配到的資料範圍是介於 1~1000 之間，則該範圍內的資料可以再細分為 1~100、101~200、...、901~1000 等的 10 個區塊來存放。

Phase 2. Load Balancing Optimization : 協調處理器(coordinator)檢查每個處理節點的資料量，若有資料歪斜產生，則決定如何重分配資料。如前一階段的例子，假設 PN1 的負載過量(overload)，則考慮從最高範圍(901~1000)的資料區塊挪出去給原分配 1001~2000 的處理節點 PN2，直到 PN1 的負擔在平衡負載的誤差條件內。而 PN2 從 PN1 接收到資料亦可能造成負載過量，則仿照 PN1，考量將最高範圍的資料區塊挪給下一個處理節點。餘此類推，直到所有處理節點的負載達到平衡。要注意的是，在此階段僅是協調者在決定資料重分配的策略，而真正資料是在下一階段才要重新傳送。

Phase 3. Redistribution: 協調處理器通知每個處理節點依據前一階段所決定的資料重分配策略，以區塊為單位重新傳送與接收資料，以達負載平衡。

Phase 4. Sorting: 每個處理節點將所分配到區塊，依序的做資料排序。

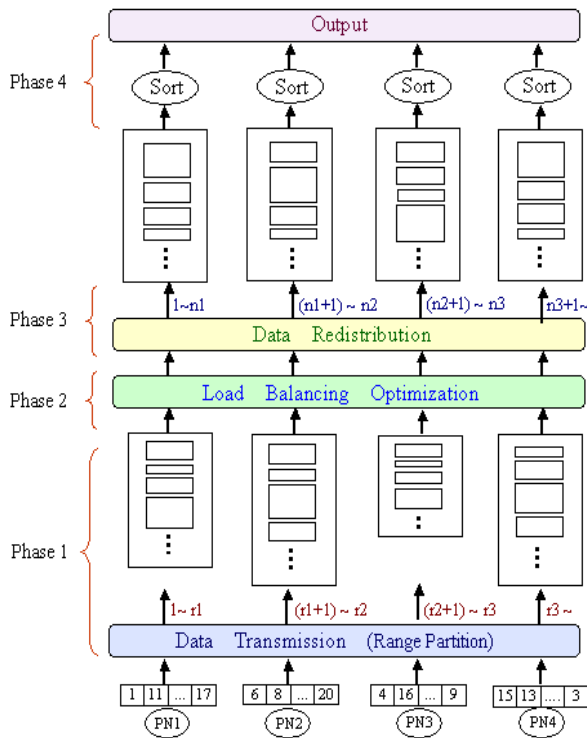


圖 5：Partitioned with Redistribution Sort

3.2 Load Balancing Optimization Sort (LBOS)

PRS 排序法是將資料傳送出去以後，再依負載平衡最佳化做資料的重分配。而 LBOS 排序法則是希望在資料傳送之前就決定好該如何分配，以避免資料再次的重分配。其演算過程如圖 6 所示，亦分四個階段完成，詳述如下：

Phase 1. Buckets Counting : 每個處理節點讀取其所擁有之局部資料(local data)，並統計每區塊之資料量，然後將此資訊傳遞給協調處理器。區塊的資料分配與 PRS 的分配方式是相同的。

Phase 2. Load Balancing Optimization : 協調處理器接收各處理節點的統計訊息後，累計每個資料區塊的資料量。之後依資料區塊的範圍順序，依序決定每個處理節點該分配那些資料區塊，以達負載平衡。要注意的是，與連結(join)運算之動態平衡不同的地方為，資料區塊不能排序，只以區塊範圍的順序依序分配，當一個處理節點分配滿了，再分配給下一個處理節點。

Phase 3. Data Partitioned : 協調處理器通知每個處理節點依據前一階段所決定的資料分配策略，傳送與接收資料，以達負載平衡。

Phase 4. Sorting: 每個處理節點將所分配到區塊的資料一一的做排序。

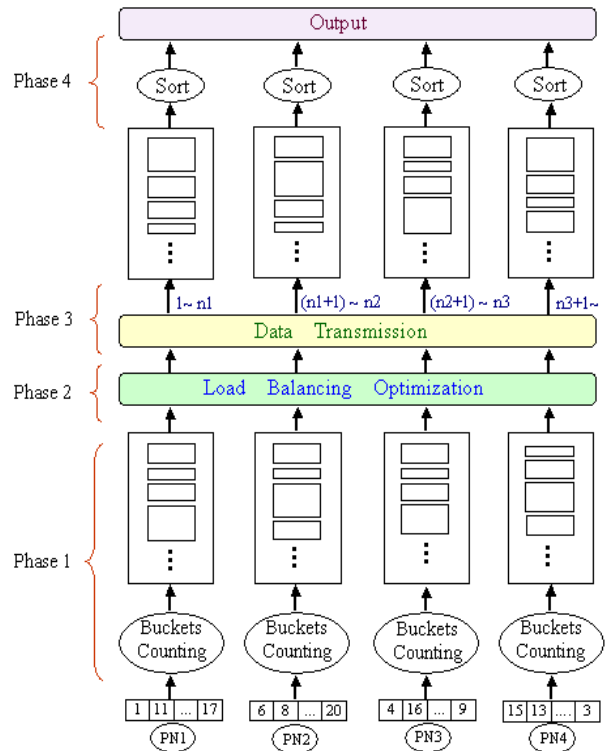


圖 6：Load Balancing Optimization Sort

4 效能分析

為了檢驗所提新的平行排序演算法對解決資料歪斜的成效，接下來我們設計了一系列的實驗，模擬平行排序的執行過程，以了解資料歪斜對排序的影響。

4.1 實驗模型

在我們的實驗裡，一開始是假設資料庫的資料是均勻的分佈在每個處理節點裡，亦即每個處理節點存放著約相同的資料量。當資料需要做排序時，依照範圍來分配的資料，就可能有資料歪斜的產生。假設排序過程所做的區塊分配就已有資料歪斜的情形，而每個區塊的資料量可以 Zipf-like[22]的分配方式來決定，其公式如下：

$$|B_i| = \frac{|R|}{i^{Z_b} \sum_{j=1}^N 1/j^{Z_b}}$$

公式中的 B_i 為第 i 區塊裡的資料量， R 為要做排序的總資料量， Z_b 為資料歪斜的程度(介於 0 與 1 之間)， N 則為總區塊數目。

下面的參數則是用來做效能的分析，類似設定亦採用於[8][9][10][11][12]。

• 負載參數

$|R|$ ：要做排序的總資料量(筆)。

Z_b ：資料歪斜的程度(0.0~1.0)。

t ：每筆資料的大小(Bytes)。

$|P_{max}|$ ：每一階段的執行前，處理節點所擁有最大之資料筆數。

$|P_{max_io}|$ ：每一階段的執行過程，處理節點所需最大的 I/O 資料筆數。

$|P_{max_comm.}|$ ：每一階段的執行過程，處理節點所需傳送與接收的最大資料筆數。

• 系統參數

n ：處理節點數。

μ ：處理器的速度(MIPS)。

w_{io} ：I/O 的速度(Mbytes/sec)。

w_{comm} ：處理節點間資料傳遞的頻寬(Mbytes/sec)。

I_{cpu} ：處理器平均處理每一筆資料排序演算所需的指令長度(instructions)。

當排序進行的每一階段雖然可能會需要 I/O 讀取與寫入資料的時間、處理器的運算時間、甚至資料的傳遞時間，但它們幾乎可以同步的進行，也就是該階段執行所需的耗費時間，幾乎可以用 I/O、CPU、或資料傳遞三者中速度最慢的來決定。接下來我們將以 Parallel Partitioned Sort(PPS)、Partitioned with Redistribution Sort(PRS)與 Load Balancing Optimization Sort(LBOS)來做

比較，如此它們演算所需的時間函數可以表示如下：

- PPS：此法大致可以分為資料分配與執行兩個階段，而各階段所需的執行時間分別以 $T_{partition}$ 與 T_{sort} 來表示，若 T_{pps} 為總執行時間，如此：

$$T_{pps} = T_{partition} + T_{sort}$$

$$T_{partition} = \text{Max}(T_{io}, T_{cpu}, T_{communication})$$

/*Max 為取最大值函數/

$$T_{io} = |P_{max_io}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

$$T_{communication} = |P_{max_comm.}| \cdot t/w_{comm}$$

$$T_{sort} = \text{Max}(T_{io}, T_{cpu})$$

$$T_{io} = |P_{max}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

- PRS：有四個階段，其中第三階段，Load Balancing Optimization，只需處理器運算的時間，沒有 I/O 或資料傳遞的必要，當要排序的資料量相當大時，此階段所需的時間相對其它階段來說將會相當的小，我們可以近似 0 來看待。如此：

$$T_{PRS} = T_{phase1} + T_{phase2} + T_{phase3} + T_{phase4}$$

$$T_{phase1} = \text{Max}(T_{io}, T_{cpu}, T_{communication})$$

$$T_{io} = |P_{max_io}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

$$T_{communication} = |P_{max_comm.}| \cdot t/w_{comm}$$

$$T_{phase2} \approx 0$$

$$T_{phase3} = \text{Max}(T_{io}, T_{cpu}, T_{communication})$$

$$T_{io} = |P_{max_io}| \cdot t/w_{io}$$

$$T_{cpu} \approx 0$$

$$T_{communication} = |P_{max_comm.}| \cdot t/w_{comm}$$

$$T_{sort} = \text{Max}(T_{io}, T_{cpu})$$

$$T_{io} = |P_{max}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

- LBOS：亦有四個階段，但第一階段只是在計算區塊大小，所以只需讀資料，不寫入資料，且只傳送區塊資訊給協調者，傳遞之資料量相對來說亦相當的小，可設為趨近於 0。如此：

$$T_{LBOS} = T_{phase1} + T_{phase2} + T_{phase3} + T_{phase4}$$

$$T_{phase1} = \text{Max}(T_{io}, T_{cpu}, T_{communication})$$

$$T_{io} = |P_{max}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

$$T_{communication} \approx 0$$

$$T_{phase2} \approx 0$$

$$T_{phase3} = \text{Max}(T_{io}, T_{cpu}, T_{communication})$$

$$T_{io} = |P_{max_io}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

$$T_{communication} = |P_{max_comm.}| \cdot t/w_{comm}$$

$$T_{sort} = \text{Max}(T_{io}, T_{cpu})$$

$$T_{io} = |P_{max}| \cdot t/w_{io}$$

$$T_{cpu} = |P_{max}| \cdot I_{cpu}/\mu$$

而實驗中我們所用到的參數值如下表，實驗結果則說明於後。

參數項目	參數值	備註
處理節點數(n)	16 ~ 256	
資料歪斜程度(Z_b)	0.0 ~ 1.0	
區塊數(N)	2560	
處理器速度 (μ)	2300MIPS	Intel P4 1.2GHz
I/O 頻寬 (w_{io})	80MB/sec	Ultra2Wide SCSI
資料傳遞頻寬 (w_{comm})	132MB/sec	
排序處理指令數(I_{cpu})	1,000 instructions	
總資料筆數($ R $)	10,000,000	
每筆資料大小(t)	500Bytes	

4.2 資料歪斜程度的影響(Effect of Bucket Skew)

為檢驗資料歪斜程度對平行排序的影響，我們變化區塊中歪斜程度從 0.0 到 1.0，而 0.0 所代表的意義為資料是均衡分配，每個區塊有著相同的資料量；1.0 則表示資料有強烈集中在部份區塊的趨勢。我們依各種不同數目的處理節點做了實驗觀察，但限於篇幅，僅就將處理節點數為 64 為代表，結果呈現於圖 7。

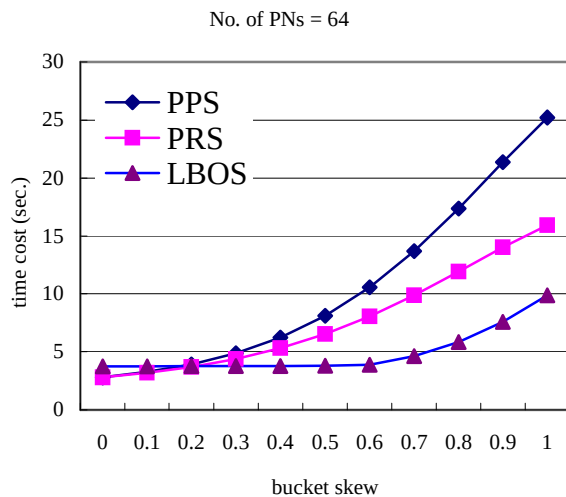


圖 7：Effect of bucket skew

由圖 7 中可以看出，當歪斜程度在 0.2 以下時，PPS 與 PRS 的表現幾乎是一樣的，那是因為

我們設定當任一處理節點的資料超過平均負載的 10% 以上時，PRS 才要做資料的重分配。當資料歪斜程度在 0.2 以下時，PRS 還未達到必須資料重分配的條件，因此 PRS 與 PPS 有著相同的表現。而在同樣的條件下，LBOS 的表現就較為遜色，但差距並不大，那是因為 LBOS 在執行的第一階段多做了一次資料的讀取，以取得區塊大小的資訊，做為負載平衡最佳化的計算。而當資料歪斜程度大於 0.2 以上時，平行排序的操作受到資料分配不均衡的影響逐漸顯現，而負載平衡最佳化的成效開始起了作用。PRS 因做了資料重分配，處理節點的負載較為平均，因此表現逐漸優於未做平衡最佳化的 PPS。當資料歪斜程度達到 1.0 時，PRS(15.93 秒)較 PPS(25.23)所需的執行時間，有著約 37.1% 的改善。再觀察來 LBOS 曲線的表現，在資料歪斜程度在 0.6 以前，一直是近乎水平線的進行，此表示 LBOS 在 0.6 之前可以將負載不平衡處理得很好，幾乎不受資料歪斜的影響。資料歪斜程度超過 0.6，LBOS 曲線開始緩和上揚，但總體來說，在資料歪斜程度大於 0.2 以後，LBOS 的表現就持續的優於 PPS 與 PRS。而這優勢隨著資料歪斜程度的升高而跟著擴大。到歪斜程度為 1.0 時，LBOS(9.87)比 PPS 所需的執行時間，約可有高達 60.9% 的改善。由這個實驗我們得到一個證實，當資料歪斜有可能發生時，做負載平衡最佳化，是絕對有其必要性。

4.3 處理節點數目的影響(Effect of No. of Processing Nodes)

此實驗中，處理節點數則由 16 逐步增加到 256，目的是在檢驗執行平行排序時，做動態負載平衡最佳化，其對系統擴充能力的影響。在此我們設定資料歪斜程度為 0.6，因為在上一節的實驗討論中，我們知道，在這個歪斜程度才開始對 LBOS 的效率產生影響，而實驗結果呈現於圖 8 中。

在圖 8 的實驗結果中，三個排序演算法隨著系統的擴充，其所需的執行時間都呈遞減趨勢，也就是系統的擴充，對排序的執行速度是有幫助的。但從另一方面來觀察，在處理節點數為 16 時，PRS (19.32 秒)相對於 PPS (24.72)所需的執行時間，約可以減省 21.8%。而 LBOS (14.9)相較於 PPS 則可以減省約 39.7% 的時間。但是當系統擴充至處理節點數為 256 時，PRS (3.45)相較於 PPS (4.41)，仍然是改善了約 21.8% 的時間。然而 LBOS (1.77)則較 PPS 大幅改善了約 59.9% 的執行時間。LBOS 的改善是從 39.7% 增加到 59.9%，由此可知，當系統不斷擴充，負載平衡的困難度越高時，LBOS 對系統效能所能產生的幫助就越大。

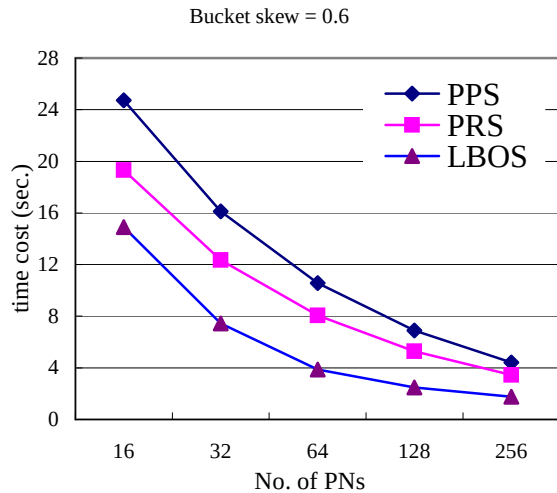


圖 8：Effect of no. of PNs

5 結論

在這個研究報告裡，我們利用了動態負載平衡的概念，解決了排序在多處理器資料庫系統中，可能遭遇到的資料歪斜問題。從研究的實驗結果，我們可以得到以下幾點的結論：

- 資料歪斜造成處理節點間負載的不平衡，其對排序所需的執行時間影響是非常大的，若不留意解決，將影響系統效能。
- 動態負載平衡對處理資料歪斜的問題，其效果是顯著的，但對資料排序的處理方式，與處理資料表格連結，並不完全相同，需要再進一步的設計，因此本研究報告提出了適用於排序之 PRS 與 LBOS 演算法。
- 平行排序的資料分配，在資料歪斜程度嚴重時，與其資料分配後再做一次的重分配，不如在分配前先讀取資料區塊之資訊，等決定好最佳之分配方式，再傳遞分配資料。因此 LBOS 明顯優於 PRS 演算法，特別是系統越大時，越顯現此演算法的優越性。

參考文獻

- [1] B. Bergsten, M. Courie, and P. Valduriez, "Overview of Parallel Architecture For Database", *The computer journal*, Vol.36, No.8, 1993, pp.734-740.
- [2] D. Bitton, H. Boral, D. DeWitt, and W. K. Wilkinson, "Parallel Algorithms for the Execution of Relational Database operations",

ACM Transactions on Database Systems, Vol.8, No.3, 1983, pp.324-353.

- [3] D. Bitton, D. J. eWitt, D. K. Hsiao, and J. Menon, "A Taxonomy of Parallel Sorting," *ACM Computing Surveys*, Vol.16, No.3, 1984, pp.287-318.
- [4] H. Boral, W. Alexander, L. Clay, G. Copeland, S. Danforth, M. Franklin, B. Hart, M. Smith, and P. Valduriez, "Prototyping Bubba, A Highly Parallel Database System," *IEEE Transaction on Knowledge and Data Engineering*, 1990, pp.4-24.
- [5] D. DeWitt, R. Gerber, G. Graefe, M. Heytens, K. Kumar, and M. Muralikrishna, "GAMMA – A High Performance Dataflow Database Machine," *In Proceedings of the Twelfth International Conference on Very Large Database*, Japan, pp.228-237, August 1986.
- [6] D. DeWitt, S. Ghandeharizadeh, D. Schneider, A. Bricker, H. Hsiao, and R. Rasmussen, "The GAMMA Database Machine Project," *IEEE Transactions on Knowledge and Data Engineering*, Vol.2, No.1, 1990, pp.44-62.
- [7] GoetzZ Graefe, "Query Evaluation Techniques for Large Databases", *ACM Computing Surveys*, Vol. 25, No. 2, June 1993, pp. 73-170.
- [8] K. A. Hua and C. Lee, "An adaptive data placement scheme for parallel database computer systems", *In Proceedings of the 16th International Conference on Very Large Database*, Brisbane, Aug.1990, pp. 493-206.
- [9] K. A. Hua and C. Lee, "Handling data skew in multiprocessor database computers using partition tuning", *In Proceedings of the 17th International Conference on Very Large Database*, Barcelona, Sept.1991, pp. 525-535.
- [10] K. A. Hua, Yu-lung Lo, and Honesty C. Young, "Considering Data Skew Factor in Multi-way Join Query Optimization for Parallel Execution," *Vldb Journal*, Volume 2, Number 3, July 1993, pp.303-330.
- [11] Kien A. Hua, Yu-lung Lo, and Honesty C. Young, "Optimizer-Assisted Load Balancing Techniques for Multicomputer Database Management Systems," *Journal of Parallel and Distributed Computing*, Volume 25, Number 1, February 15, 1995, pp. 42-57.
- [12] K. A. Hua, C. Lee, and C. M. Hua, "Dynamic Loading Balancing in Multi-computer Database Systems using Partition Tuning", *IEEE Transaction on Knowledge and Data Engineering*, Vol.7, No.6, Dec. 1995, pp.968-983.

- [13] M. Kitsuregawa, H. Tanaka, and T. Moto-oka, "Application of Hash to Database Machine and Its Architecture", *New Generation Computing*, 1983, pp.66-74.
- [14] P. Mishra and M. H. Eich, "Join Processing in relational database", *ACM Computing Surveys*, Vol .24, Vo.1, March 1992, pp. 63-113.
- [15] James P. Richardson, Hongjun Lu, Krishna Mikkilineni, "Design and Evaluation of Parallel Pipelined Join Algorithms", *In Proceedings of the ACM SIGMOD Annual Conference on Management*, 1987, pp.399-409.
- [16] D. Schneider and D. DeWitt, "A Performance Evaluation of Four Parallel Join Algorithms in a Shared-Nothing Multiprocessor Environment, " *In Proceed of ACM-SIGMOD, Portland, OR*, 1989, pp. 110-121.
- [17] M. Stonebraker, "The Case for Sharing Nothing", *IEEE Database Engineering*, Vol.9, No.1, Mar.1986, pp. 4-9.
- [18] D. Taniar and J. W. Rahayu, " Sorting in Parallel Database System", *Proceedings of The Fourth International Conference on High Performance Computing in the Asia-Pacific Region*, Vol.2, 2000, pp.830-835.
- [19] Teradata Corporation, Los Angeles, California, *Teradata DBC/1012 Data Base Computer Concepts and Facilities*, release 3.1 edition, 1988, Teradata Document C02-0001-05.
- [20] Estivill-Castro Vladimir," A Survey of Adaptive Sorting Algorithms" *ACM Computing Surveys*, Vol. 24, No. 4, Dec. 1992, pp.441-476.
- [21] J. L. Wolf, D. M. Dias, and P. S. Yu, "A Parallel Sort Merge Join Algorithm for Managing Data Skew", *IEEE Transactions Parallel and Distributed Systems*, Vol.4, No.1, 1993, pp.70-86.
- [22] G. K. Zipf, "Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology," *Reading, Addison-Wesley, Mass.*, 1949.